Day 01/30 Spring - Spring Boot

What is Spring?

Spring is a **Java framework** used to build applications easily, especially **enterprise and web applications**.

☞ It reduces boilerplate code and helps you write **clean, modular, and testable code**.

Why Spring is used?

Before Spring:

- Developers had to manage everything manually (object creation, dependencies, configs)
- Code became **tightly coupled** and hard to maintain

After Spring:

- It handles most things automatically
- Makes code **loosely coupled** and easy to scale

Core Concepts of Spring

1. Dependency Injection (DI)

☞ Spring creates objects and injects dependencies automatically.

Example:
Instead of:

```
Car car = new Car();
```

Spring does:

- Creates Car
- Injects required objects (like Engine)

✔ Benefit: Loose coupling

---

2. Inversion of Control (IoC)

☞ Control is given to Spring container

- You don’ t create objects
- Spring manages lifecycle

✔ “Spring controls your objects”

---

3. Spring Container

☞ Heart of Spring

- Creates objects (Beans)
- Manages lifecycle
- Injects dependencies

---

4. Beans

☞ Objects managed by Spring

Example:

```
@Component
class Car {}
```

---

5. Aspect-Oriented Programming (AOP)

👉 Handles cross-cutting concerns

- Logging
- Security
- Transactions

Spring Modules (Important)

- **Spring Core** → DI, IoC
- **Spring MVC** → Web apps
- **Spring Data** → DB operations
- **Spring Security** → Authentication
- **Spring Boot** → Easy setup (most used 🚀)

Why Spring Boot?

Spring Boot is built on Spring:

- No XML config
- Embedded server (Tomcat)
- Faster development

Simple Definition (for interview)

☞ *"Spring is a lightweight Java framework that provides infrastructure support for developing Java applications using Dependency Injection and Inversion of Control."*

What is **Spring Boot**?

Spring Boot is a framework built on top of Spring that helps you create **production-ready applications quickly** with **minimal configuration**.

☞ It removes complexity of traditional Spring.

Simple Definition (Interview)

☞ *"Spring Boot is an extension of Spring that simplifies application development by providing auto-configuration, embedded servers, and ready-to-use features."*

Why Spring Boot?

Before Spring Boot:

- Heavy XML configuration 😬
- Manual server setup (Tomcat)
- Dependency management was complex

After Spring Boot:

- Zero/less configuration ✅
- Embedded server ✅
- Fast development 🚀

Key Features

1. Auto Configuration

☞ Automatically configures your app based on dependencies

Example:

- Add MySQL dependency → Spring configures DB automatically

---

2. Embedded Server

☞ No need to install server separately

- Comes with:
  - Tomcat (default)
  - Jetty
  - Undertow

Run app:

```
mvn spring-boot:run
```

✔ Your app runs like a normal Java program

---

3. Starter Dependencies

☞ Predefined dependency packages

Examples:

- spring-boot-starter-web
- spring-boot-starter-data-jpa
- spring-boot-starter-security

✔ No need to add multiple dependencies manually

---

4. Production Ready Features

☞ Built-in tools

- Health check (Actuator)
- Metrics
- Logging

---

5. Opinionated Defaults

☞ Spring Boot decides best configurations for you

✔ Reduces decision fatigue

Project Structure

Typical structure:

```
src/main/java
  └── com.example.demo
      ├── controller
      ├── service
      ├── repository
      └── model

src/main/resources
  ├── application.properties
```

Basic Example

Main Class

```
@SpringBootApplication
public class MyApp {
    public static void main(String[] args) {
        SpringApplication.run(MyApp.class, args);
    }
}
```

Controller

```
@RestController
public class HelloController {

    @GetMapping("/hello")
    public String hello() {
        return "Hello World";
    }
}
```

Run → http://localhost:8080/hello

Important Annotations

- @SpringBootApplication → Main config
- @RestController → API controller
- @Autowired → Dependency injection
- @Service → Business logic
- @Repository → DB layer

Real Use Cases

- REST APIs
- Microservices
- Enterprise apps

- Backend for mobile/web apps

Spring vs Spring Boot

Feature	Spring	Spring Boot
Config	Manual	Auto
Setup	Complex	Simple
Server	External	Embedded
Speed	Slow setup	Fast 🚀

One Line Summary

☞ *Spring Boot = Spring + Auto Configuration + Embedded Server + Fast Development*

What is **IoC (Inversion of Control)?**

☞ **Inversion of Control** means:

Control of object creation and management is transferred from your code to the Spring framework (container).

Simple Understanding

❌ Without IoC (Traditional way)

You create objects manually:

```
class Engine {}

class Car {
    Engine engine = new Engine(); // tightly coupled ❌
}
```

☞ Problem:

- Car is dependent on Engine directly
- Hard to change or test

✅ With IoC (Spring way)

```
@Component
class Engine {}

@Component
class Car {
    @Autowired
    Engine engine; // Spring injects it ✅
}
```

☞ Here:

- You don’ t create Engine
- Spring creates and injects it

✔ This is IoC

Why called "Inversion"?

☞ Normally:

- You control objects

☞ In IoC:

- **Spring controls objects**

✔ Control is “inverted” (reversed)

IoC Container

☞ The component responsible is **Spring Container**

Two types:

- **BeanFactory** (basic)
- **ApplicationContext** (advanced, most used)

What IoC Container Does

- Creates objects (Beans)

- Injects dependencies
- Manages lifecycle
- Handles configuration

Key Concept: Bean

☞ A **Bean** is just an object managed by Spring

```
@Component
class Car {}
```

✔ Now `Car` is a Spring Bean

How IoC Works (Flow)

1. Spring starts application
2. Scans classes (`@Component`, `@Service`, etc.)
3. Creates objects (Beans)
4. Injects dependencies (`@Autowired`)
5. Manages lifecycle

IoC + Dependency Injection

☞ IoC is concept
☞ DI is implementation

Types of DI:

- Constructor Injection (best ✓)
- Setter Injection

- Field Injection

Real Life Example

☞ Think like this:

- You order food on an app 🍔
- You don’ t cook
- App delivers food

✔ You don’ t control cooking → control is inverted

Same:

- You don’ t create objects
- Spring gives you ready objects

Advantages

- Loose coupling ✅
- Easy testing ✅
- Better code structure ✅
- Easy maintenance ✅

Interview Definition

☞ *“IoC is a design principle where the control of object creation and dependency management is transferred to the Spring container.”*

One Line Summary

☞ *You don’t create objects → Spring creates and injects them.*

What is **Dependency Injection (DI)?**

☞ **Dependency Injection** means:

Providing required objects (**dependencies**) to a class **from outside**, instead of creating them inside the class.

Simple Example

❌ Without DI (Bad Practice)

```
class Engine {}

class Car {
    Engine engine = new Engine(); // tightly coupled ❌
}
```

☞ Problem:

- Car creates Engine
- Hard to replace or test

✅ With DI (Spring Way)

```
@Component
class Engine {}

@Component
class Car {
    Engine engine;

    @Autowired
    public Car(Engine engine) {
        this.engine = engine;
```

```
    }
}
```

☞ Here:

- Spring provides `Engine` to `Car`
- `Car` does not create it

✔ This is Dependency Injection

Key Idea

☞ **Class should not create dependencies → it should receive them**

Types of Dependency Injection

1. Constructor Injection (Best ✓✓)

```
@Component
class Car {
    private Engine engine;

    public Car(Engine engine) {
        this.engine = engine;
    }
}
```

✔ Advantages:

- Mandatory dependencies
- Immutable
- Best for testing

---

2. Setter Injection

```
@Component
class Car {
    private Engine engine;

    @Autowired
    public void setEngine(Engine engine) {
        this.engine = engine;
    }
}
```

✔ Used when dependency is optional

---

3. Field Injection (Not recommended ✕)

```
@Component
class Car {
    @Autowired
    private Engine engine;
}
```

✕ Problems:

- Hard to test
- Not clean

How Spring DI Works

1. Spring scans classes (@Component, @Service)
2. Creates objects (Beans)
3. Finds dependencies
4. Injects them automatically

Real-Life Example

👉 Think:

- You go to restaurant 🍽️
- You don’ t cook food
- Chef prepares and serves

✔ You receive dependency (food)

Same in Spring:

- You receive objects from Spring

DI vs IoC

Concept	Meaning
IoC	Control is given to Spring
DI	Way to achieve IoC

👉 DI is **implementation of IoC**

Advantages

- Loose coupling ✅
- Easy testing (mocking) ✅
- Better maintainability ✅
- Cleaner code ✅

Important Annotations

- @Component
- @Service
- @Repository
- @Autowired
- @Qualifier (when multiple beans)

Interview Definition

☞ *"Dependency Injection is a design pattern where dependencies are provided by an external system (like Spring container) instead of being created inside the class."*

One Line Summary

☞ *Don't create objects → get them injected by Spring.*

Here is a **complete working mini project** 👇

✅ Project: Engine-Car

(Using @Component + @Autowired)

📁 Structure

```
com.example.demo
|
├── component
|     └── Engine.java
|
├── service
|     └── CarService.java
|
├── controller
|     └── CarController.java
|
└── DemoApplication.java
```

---

1 Main Class

```
package com.example.demo;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class DemoApplication {
    public static void main(String[] args) {
        SpringApplication.run(DemoApplication.class, args);
    }
}
```

---

2 Engine (Generic Component)

```
package com.example.demo.component;

import org.springframework.stereotype.Component;

@Component
public class Engine {

    public String startEngine() {
        return "Engine Started 🚗";
    }
}
```

👉 This is a **generic bean using @Component**

---

3 Service (Using @Autowired)

```
package com.example.demo.service;

import com.example.demo.component.Engine;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

@Service
public class CarService {

    @Autowired   // Dependency Injection here
    private Engine engine;

    public String drive() {
        return engine.startEngine() + " → Car is moving";
    }
}
```

---

4 Controller

```
package com.example.demo.controller;
```

```
import com.example.demo.service.CarService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class CarController {

    @Autowired
    private CarService carService;

    @GetMapping("/drive")
    public String driveCar() {
        return carService.drive();
    }
}
```

▶ Run Project

```
mvn spring-boot:run
```

🔗 Test API

http://localhost:8080/drive

✅ Output:

```
Engine Started → Car is moving
```

🔥 What's Happening Internally

Step-by-step:

1. Spring scans `@Component`
2. Creates `Engine` object (Bean)
3. Sees `@Autowired` in `CarService`

4. Injects Engine into CarService
5. Injects CarService into CarController

🎯 Where Concepts Used

Concept	Code
IoC	Spring creates all objects
DI	@Autowired
Component	@Component Engine
Layering	Controller → Service

⚠ Important Note

👉 This uses **Field Injection (@Autowired)**

✓ Good for learning
✗ Not best practice in real projects

✅ Best Practice Version (Short)

```
@Service
public class CarService {

    private final Engine engine;

    public CarService(Engine engine) {
        this.engine = engine;
    }
}
```

👉 No @Autowired needed ✅

Final Summary

☞ @Component → creates bean

☞ @Autowired → injects bean

☞ Spring handles everything (IoC)

What is **Dependency Injection (DI)?**

☞ **Dependency Injection** means:

Providing required objects (**dependencies**) to a class **from outside**, instead of creating them inside the class.

Simple Example

❌ Without DI (Bad Practice)

```
class Engine {}

class Car {
    Engine engine = new Engine(); // tightly coupled ❌
}
```

☞ Problem:

- Car creates Engine
- Hard to replace or test

✅ With DI (Spring Way)

```
@Component
class Engine {}

@Component
class Car {
    Engine engine;

    @Autowired
    public Car(Engine engine) {
        this.engine = engine;
```

```
    }
}
```

☞ Here:

- Spring provides `Engine` to `Car`
- `Car` does not create it

✓ This is Dependency Injection

Key Idea

☞ **Class should not create dependencies → it should receive them**

Types of Dependency Injection

1. Constructor Injection (Best ✓)

```
@Component
class Car {
    private Engine engine;

    public Car(Engine engine) {
        this.engine = engine;
    }
}
```

✓ Advantages:

- Mandatory dependencies
- Immutable
- Best for testing

---

2. Setter Injection

```
@Component
class Car {
    private Engine engine;

    @Autowired
    public void setEngine(Engine engine) {
        this.engine = engine;
    }
}
```

✓ Used when dependency is optional

---

3. Field Injection (Not recommended ✕)

```
@Component
class Car {
    @Autowired
    private Engine engine;
}
```

✕ Problems:

- Hard to test
- Not clean

How Spring DI Works

1. Spring scans classes (@Component, @Service)
2. Creates objects (Beans)
3. Finds dependencies
4. Injects them automatically

Real-Life Example

👉 Think:

- You go to restaurant 🍽️
- You don’ t cook food
- Chef prepares and serves

✓ You receive dependency (food)

Same in Spring:

- You receive objects from Spring

DI vs IoC

Concept	Meaning
IoC	Control is given to Spring
DI	Way to achieve IoC

👉 DI is **implementation of IoC**

Advantages

- Loose coupling ✅
- Easy testing (mocking) ✅
- Better maintainability ✅
- Cleaner code ✅

Important Annotations

- @Component
- @Service
- @Repository
- @Autowired
- @Qualifier (when multiple beans)

Interview Definition

☞ *"Dependency Injection is a design pattern where dependencies are provided by an external system (like Spring container) instead of being created inside the class."*

One Line Summary

☞ *Don't create objects → get them injected by Spring.*

Next Step (Day 2 suggestion)

☞ **Spring Bean Lifecycle (very important interview topic)**
or
☞ **@Component vs @Service vs @Repository**

Just tell 👆